

SPESIFIKASI PROGRAM APLIKASI "KASIRKU"

Nama Aplikasi: KasirKu **Folder Proyek:** **serkom** **Jenis:** Aplikasi Web Kasir (*Point of Sale*) Sederhana **Framework:** Laravel 12 (pola MVC) **PHP:** 8.2 ke atas **Database:** MySQL **Nama Database:** **db_kasirku** **Tampilan:** Blade, Bootstrap 5.3, dan JavaScript

1. DESKRIPSI APLIKASI

KasirKu adalah aplikasi web kasir (*Point of Sale* sederhana) untuk toko/retail kecil.

Aplikasi digunakan untuk:

- Mengelola data produk yang meliputi nama, harga, dan stok.
- Mencatat transaksi penjualan dengan lebih dari satu item dalam satu nota.
- Mengurangi stok produk secara otomatis setiap transaksi berhasil disimpan.
- Menampilkan riwayat transaksi.
- Menampilkan rangkuman penjualan.

Aplikasi **tidak menggunakan login**, sehingga seluruh halaman dapat langsung diakses oleh kasir.

Data awal berupa contoh produk disediakan melalui **Seeder**, yaitu **ProdukSeeder**. Seeder juga menyediakan contoh produk dengan stok 0 sebagai contoh kondisi stok habis.

Pembagian tugas berdasarkan pola **MVC (Model – View – Controller)**:

Model

- **Tugas:** Mewakili tabel dan relasi database menggunakan Eloquent ORM.
- **File:** **app/Models/Produk.php**, **Transaksi.php**, **DetailTransaksi.php**

View

- **Tugas:** Menampilkan antarmuka aplikasi menggunakan Blade.
- **File:** **resources/views/layouts/**, **produk/**, **transaksi/**

Controller

- **Tugas:** Menangani logika bisnis, CRUD produk, perhitungan, stok, dan transaksi.
- **File:** **app/Http/Controllers/ProdukController.php**, **TransaksiController.php**

Route

- **Tugas:** Memetakan URL ke method pada controller.
- **File:** **routes/web.php**

Migration

- **Tugas:** Membuat struktur tabel database.
- **File:** **database/migrations/**

Seeder

- **Tugas:** Mengisi data awal produk.

- **File:** `database/seeder/ProdukSeeder.php`

2. DAFTAR FITUR

2.1 Daftar Produk

Menampilkan data produk dalam bentuk tabel yang berisi:

- Nama produk.
- Harga.
- Stok.
- Status stok.

Ketentuan:

- Produk terbaru ditampilkan di bagian atas.
- Maksimal **10 data per halaman**.
- Produk dengan stok 0 diberi badge "**Habis**".

2.2 Tambah Produk

Menyediakan form untuk menambahkan produk baru.

Validasi input:

- Nama produk wajib diisi.
- Nama produk harus unik.
- Harga harus berupa bilangan bulat.
- Harga minimal 0.
- Stok harus berupa bilangan bulat.
- Stok minimal 0.

2.3 Ubah Produk

Menyediakan form untuk mengubah data produk.

Validasi yang digunakan sama dengan form tambah produk:

- Nama produk wajib diisi.
- Nama produk harus unik.
- Harga berupa bilangan bulat minimal 0.
- Stok berupa bilangan bulat minimal 0.

Pada pengecekan nama unik, produk yang sedang diedit tidak dihitung sebagai duplikat.

2.4 Hapus Produk

Produk dapat dihapus melalui tombol hapus dengan konfirmasi terlebih dahulu.

Ketentuan:

- Produk yang belum pernah digunakan dalam transaksi dapat dihapus.

- Produk yang sudah tercatat dalam riwayat transaksi tidak dapat dihapus.
 - Jika produk tidak dapat dihapus, sistem menampilkan pesan error yang mudah dipahami.
-

2.5 Form Transaksi Penjualan

Form transaksi digunakan untuk membuat transaksi penjualan baru.

Fitur yang tersedia:

- Memilih produk.
 - Memasukkan jumlah pembelian.
 - Menambahkan lebih dari satu baris item.
 - Menghapus baris item.
 - Hanya menampilkan produk yang memiliki stok lebih dari 0.
 - Membatasi jumlah pembelian sesuai stok yang tersedia.
 - Menghitung subtotal secara otomatis di browser.
 - Menghitung total bayar secara otomatis sebagai preview.
-

2.6 Perhitungan Bisnis

Perhitungan bisnis dipisahkan ke dalam tiga fungsi pada `TransaksiController`:

- `hitungSubtotal($harga, $jumlah)` Menghitung subtotal setiap item dengan rumus:
Subtotal = Harga × Jumlah
 - `hitungTotalBayar($subtotal)` Menjumlahkan seluruh subtotal item menjadi total pembayaran.
 - `kurangiStok($produk_id, $jumlah)` Mengurangi stok produk berdasarkan jumlah pembelian dan menolak transaksi apabila stok tidak mencukupi atau hasil stok menjadi negatif.
-

2.7 Penyimpanan Transaksi Aman

Penyimpanan transaksi menggunakan `DB::transaction`.

Data yang diproses dalam satu transaksi database:

- Header transaksi.
- Detail transaksi.
- Pengurangan stok produk.

Jika terjadi kegagalan, misalnya stok tidak mencukupi, seluruh perubahan akan dibatalkan menggunakan mekanisme **rollback**.

Dengan demikian, tidak akan terjadi data transaksi yang tersimpan sebagian.

2.8 Riwayat Transaksi

Halaman riwayat transaksi menampilkan:

- Tanggal transaksi.
- Jumlah item.

- Total bayar.
- Maksimal **10 transaksi per halaman**.

Halaman ini juga menampilkan rangkuman:

- **Total Penjualan**
 - **Jumlah Transaksi**
 - **Total Unit Terjual**
-

2.9 Detail Transaksi / Struk

Setiap transaksi dapat dibuka untuk melihat detail struk.

Informasi yang ditampilkan:

- Nomor transaksi.
 - Tanggal transaksi.
 - Nama produk.
 - Harga satuan.
 - Jumlah pembelian.
 - Subtotal setiap produk.
 - Total bayar.
-

2.10 Validasi dan Pesan Error

Validasi dilakukan pada sisi server menggunakan:

```
$request->validate()
```

Sistem memberikan pesan sukses maupun error yang mudah dipahami.

Contoh pesan ketika stok tidak mencukupi:

```
"Stok 'Beras 5kg' tidak mencukupi. Sisa stok: 3."
```

3. STRUKTUR DATABASE

Database yang digunakan adalah **db_kasirku** dengan MySQL.

Database memiliki tiga tabel utama:

- **produk**
- **transaksi**
- **detail_transaksi**

Tabel bawaan Laravel seperti **users**, **cache**, **jobs**, dan tabel sejenisnya dapat ikut dibuat oleh migration, tetapi tidak digunakan sebagai fitur utama aplikasi.

3.1 Tabel **produk**

Tabel **produk** merupakan tabel master yang menyimpan data barang.

Kolom	Tipe	Keterangan
id	bigint	Primary key, auto increment
nama_produk	varchar(255)	Nama produk
harga	bigint	Harga satuan dalam rupiah
stok	integer	Jumlah persediaan
created_at	timestamp	Waktu pembuatan data
updated_at	timestamp	Waktu perubahan data

Ketentuan:

- **nama_produk** divalidasi agar unik.
- **harga** minimal 0.
- **stok** minimal 0.

3.2 Tabel **transaksi**

Tabel **transaksi** merupakan tabel header atau kepala nota.

Kolom	Tipe	Keterangan
id	bigint	Primary key, auto increment
tanggal	datetime	Waktu transaksi dibuat
total_bayar	bigint	Total seluruh subtotal
created_at	timestamp	Waktu pembuatan data
updated_at	timestamp	Waktu perubahan data

3.3 Tabel **detail_transaksi**

Tabel **detail_transaksi** menyimpan rincian item dari setiap transaksi.

Kolom	Tipe	Keterangan
id	bigint	Primary key, auto increment
transaksi_id	bigint	Foreign key ke transaksi.id
produk_id	bigint	Foreign key ke produk.id
jumlah	integer	Jumlah barang yang dibeli
subtotal	bigint	Hasil harga × jumlah
created_at	timestamp	Waktu pembuatan data
updated_at	timestamp	Waktu perubahan data

Ketentuan foreign key:

- `transaksi_id` menggunakan **ON DELETE CASCADE**.
- `produk_id` menggunakan **ON DELETE RESTRICT**.

Dengan demikian, transaksi yang dihapus akan menghapus detailnya, sedangkan produk yang sudah digunakan dalam transaksi tidak dapat dihapus.

3.4 Relasi Antar Tabel

Relasi database:

```
transaksi (1) ———< (n) detail_transaksi >—— (1) produk
```

Artinya:

- Satu transaksi memiliki banyak detail transaksi.
- Satu produk dapat muncul pada banyak detail transaksi.
- `detail_transaksi` menjadi tabel penghubung antara `transaksi` dan `produk`.

3.5 Relasi Eloquent

Relasi	Tipe	Method	Foreign Key
Transaksi → Detail	<code>hasMany</code>	<code>Transaksi::detailTransaksis()</code>	<code>detail_transaksi.transaksi_id</code>
Produk → Detail	<code>hasMany</code>	<code>Produk::detailTransaksis()</code>	<code>detail_transaksi.produk_id</code>
Detail → Transaksi	<code>belongsTo</code>	<code>DetailTransaksi::transaksi()</code>	<code>detail_transaksi.transaksi_id</code>
Detail → Produk	<code>belongsTo</code>	<code>DetailTransaksi::produk()</code>	<code>detail_transaksi.produk_id</code>

4. ROUTE YANG DIGUNAKAN

Semua route terdapat pada file:

```
routes/web.php
```

Route produk menggunakan prefix `/produk`, sedangkan route transaksi menggunakan prefix `/transaksi`.

No	Method	URL	Controller@Method	Nama Route	Fungsi
1	GET	<code>/</code>	<code>ProdukController@index</code>	-	Beranda / daftar produk
2	GET	<code>/produk</code>	<code>ProdukController@index</code>	<code>produk.index</code>	Daftar produk
3	GET	<code>/produk/create</code>	<code>ProdukController@create</code>	<code>produk.create</code>	Form tambah

No	Method	URL	Controller@Method	Nama Route	Fungsi
					produk
4	POST	/produk	ProdukController@store	produk.store	Simpan produk baru
5	GET	/produk/{produk}/edit	ProdukController@edit	produk.edit	Form ubah produk
6	PUT	/produk/{produk}	ProdukController@update	produk.update	Simpan perubahan produk
7	DELETE	/produk/{produk}	ProdukController@destroy	produk.destroy	Hapus produk
8	GET	/transaksi/create	TransaksiController@create	transaksi.create	Form transaksi
9	POST	/transaksi	TransaksiController@store	transaksi.store	Simpan transaksi + kurangi stok
10	GET	/transaksi	TransaksiController@index	transaksi.index	Riwayat transaksi + rangkuman
11	GET	/transaksi/{transaksi}	TransaksiController@show	transaksi.show	Detail transaksi / struk

Catatan: Route `/transaksi/create` harus ditulis sebelum `/transaksi/{transaksi}` agar kata `create` tidak dianggap sebagai ID transaksi.

5. FLOWCHART ALUR TRANSAKSI

Alur transaksi dibaca dari atas ke bawah.

5.1 Alur Menyimpan Transaksi Penjualan

```

[1] MULAI
    |
    v
Kasir membuka halaman /transaksi/create
    |
    v
[2] Kasir memilih produk dan jumlah beli
    |
    v
Browser menampilkan preview subtotal dan total
serta membatasi jumlah sesuai stok
  
```

```

|
v
[3] Kasir klik "Simpan Transaksi"
|
v
Data dikirim melalui POST /transaksi
|
v
[4] KEPUTUSAN: Apakah input valid?
|
+---- TIDAK ----> Kembali ke form + pesan error
|
|                               |
|                               v
|                               SELESAI
|
+---- YA -----> Lanjut ke langkah 5
|
|                               v

[5] Mulai DB::transaction
|
v
Buat header transaksi
tanggal = sekarang
total_bayar = 0
|
v
[6] Ambil item berikutnya dari keranjang
|
v
Hitung subtotal dengan:
hitungSubtotal(harga, jumlah)
|
v
[7] Simpan detail_transaksi
produk_id
jumlah
subtotal
|
v
[8] KEPUTUSAN: Apakah stok cukup?
|
+---- TIDAK ----> Lempar Exception
|
|                               |
|                               v
|                               DB ROLLBACK
|
|                               v
|                               Kembali ke form
|                               + pesan error
|
|                               v
|                               SELESAI
|
+---- YA -----> Lanjut ke langkah 9
|
|                               v

[9] Kurangi stok:
stok = stok - jumlah

```

```

    |
    v
Tambahkan subtotal ke total:
hitungTotalBayar(subtotal)
    |
    v
[10] KEPUTUSAN:
Masih ada item lain?
    |
    +---- YA -----> Kembali ke langkah 6
    |
    +---- TIDAK -----> Lanjut ke langkah 11
                           |
                           v
[11] Update total_bayar pada header
    |
    v
COMMIT
    |
    v
[12] Redirect ke /transaksi
dengan pesan sukses
    |
    v
[13] SELESAI
Riwayat transaksi dan rangkuman tampil

```

5.2 Alur Saat Terjadi Kegagalan

```

[Exception]
Stok tidak mencukupi / terjadi kegagalan
    |
    v
[DB ROLLBACK]
    |
    v
Header transaksi dibatalkan
Detail transaksi dibatalkan
Perubahan stok dibatalkan
    |
    v
Kembali ke form transaksi
    |
    v
Tampilkan pesan error
    |
    v
User memperbaiki input
    |
    v
Transaksi dapat dicoba kembali

```

Contoh pesan error:

```
"Stok 'Beras 5kg' tidak mencukupi.  
Sisa stok: 3."
```

5.3 Alur Perhitungan Riwayat

```
[Tabel riwayat transaksi dimuat]  
|  
+----> SUM(total_bayar)  
|      |  
|      v  
|      Total Penjualan  
|  
+----> COUNT(transaksi)  
|      |  
|      v  
|      Jumlah Transaksi  
|  
+----> SUM(detail_transaksi.jumlah)  
|      |  
|      v  
|      Total Unit Terjual
```

Rangkuman yang ditampilkan:

- **Total Penjualan** = `SUM(total_bayar)`
- **Jumlah Transaksi** = `COUNT(transaksi)`
- **Total Unit Terjual** = `SUM(detail_transaksi.jumlah)`

6. CARA MENJALANKAN APLIKASI

6.1 Membuat Database

Pastikan MySQL pada XAMPP sudah berjalan.

Jalankan:

```
mysql -u root -e "CREATE DATABASE IF NOT EXISTS db_kasirku CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;"
```

6.2 Mengatur File `.env`

Sesuaikan konfigurasi database:

```
DB_DATABASE=db_kasirku  
DB_USERNAME=root
```

```
DB_PASSWORD=
```

6.3 Menjalankan Migration dan Seeder

Jalankan:

```
php artisan migrate --seed
```

Perintah tersebut digunakan untuk:

- Membuat tabel database melalui migration.
 - Mengisi data awal produk melalui **ProdukSeeder**.
-

6.4 Menjalankan Server

Jalankan:

```
php artisan serve
```

Kemudian buka:

```
http://127.0.0.1:8000
```

7. STANDAR KODE

Aplikasi mengikuti standar penulisan kode **PSR-12**.

Ketentuan utama:

- Menggunakan namespace yang sesuai.
- Kurung kurawal `{ }` ditulis pada baris baru sesuai standar.
- Menggunakan indentasi 4 spasi.
- File diakhiri dengan baris baru.
- Penulisan kode dibuat konsisten dan mudah dibaca.

Standar kode dapat diperiksa menggunakan:

```
./vendor/bin/pint --test
```

8. KOMENTAR PADA KODE

Komentar edukatif hanya digunakan pada bagian-bagian penting agar kode tetap mudah dipahami dan tidak terlalu penuh komentar.

Komentar dapat diberikan pada:

- Fungsi perhitungan bisnis.
- Validasi stok.
- Relasi Eloquent.
- Proses penyimpanan transaksi.
- Penggunaan `DB::transaction`.
- Proses rollback ketika terjadi kegagalan.

Tujuan komentar adalah membantu menjelaskan **logika penting aplikasi**, bukan menjelaskan setiap baris kode.

9. RINGKASAN APLIKASI

KasirKu merupakan aplikasi kasir sederhana berbasis **Laravel 12** dengan pola **MVC**.

Fungsi utama aplikasi meliputi:

- CRUD produk.
- Validasi data produk.
- Transaksi dengan banyak item.
- Perhitungan subtotal dan total pembayaran.
- Pengurangan stok otomatis.
- Validasi stok.
- Database transaction dan rollback.
- Riwayat transaksi.
- Detail transaksi atau struk.
- Rangkuman total penjualan.
- Seeder untuk data awal produk.

Aplikasi menggunakan **Laravel, PHP, MySQL, Blade, Bootstrap 5.3, dan JavaScript** serta menerapkan struktur database yang terdiri dari tabel `produk`, `transaksi`, dan `detail_transaksi`.